React JS

Agenda

- Routing
- HTTP Requests using Axios
- useEffect hook
- UseRef hook
- Questions!

Routing

Routing is the capacity to show different pages to the user. That means the user can move between different parts of an application by entering a URL or clicking on an element.

React at its core is a very simple library, and it does not dictate anything about routing, so you need to install it first.

npm install react-router

https://reactrouter.com/start/library/installation

Routing

<BrowserRouter>

A <BrowserRouter> stores the current location in the browser's address bar using clean URLs and navigates using the browser's built-in history stack.

<Routes>

Rendered anywhere in the app, <Routes> will match a set of child routes from the current location, Whenever the location changes, <Routes> looks through all its child routes to find the best match and renders that branch of the UI.

<Route>

Routes are perhaps the most important part of a React Router app. They couple URL segments to components, each route should specify 2 mandatory props the path and element which takes JSX component.

Routing

Hazem Alsaqaan

```
<BrowserRouter>
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/products" case element={<Products />} />
    </Routes>
  </BrowserRouter>
```

Routing

- To Define **<u>not found</u>** page use the * for the path

```
<Route path="*" element={<h1>Not found page</h1>} />
```

- To Define **<u>dynamic segments</u>** page use the **:property** for the path

```
<Route path="/details/:id" case element={<Details />} />
```

Routing-Link

A <Link> is an element that lets the user navigate to another page by clicking or tapping on it.a <Link> renders an accessible <a> element with a real href that points to the resource it's linking to.

Routing - Active links

Most web apps have persistent navigation sections at the top of the UI, the sidebar, and often multiple levels. Styling the active navigation items so the user knows where they are

Using NavLink: is a special kind of <Link> that knows whether or not it is "active" or "pending"

Hazem Alsaqaan

```
<NavLink
  to="/"
  className={({ isActive, isPending }) => {
    return isActive ? "active" : isPending ? "pending" : "";
  }}
>
  Home
</NavLink>
```

Routing-Routing hooks

- **useNavigate:**
  - This hook tells you everything you need to know about a page navigation, can use the navigate function to go from route to other like **navigate("/products")**
- **useParams :**
  - The useParams hook returns an object of key/value pairs of the dynamic params from the current URL that were matched by the route.
- **useLocation:**
  - This hook returns the current location object.
- **useSearchParams:**
  - The useSearchParams hook is used to read and modify the query string in the URL for the current location.
  - const [searchParams, setSearchParams] = **useSearchParams**();
  - **searchParams.get('paramName')**

Routing- Having Layout [Extra]

We will need to wrap the routes with a parent wrapper that holds the layout element with the navbar and footer for example.

Hazem Alsaqaan

```
<BrowserRouter>
      <Routes>
        <Route element={<Layout />}>
              <Route path="/" element={<Home />} />
        </Route>
        <Route path="/about-us" element={<AboutUs />} />
      </Routes>
</BrowserRouter>
```

Routing- Having Layout [Extra]

And inside the layout component we can use the navbar and the <Outlet /> element from react-router-dom to be as a placeholder for the components that match the route.

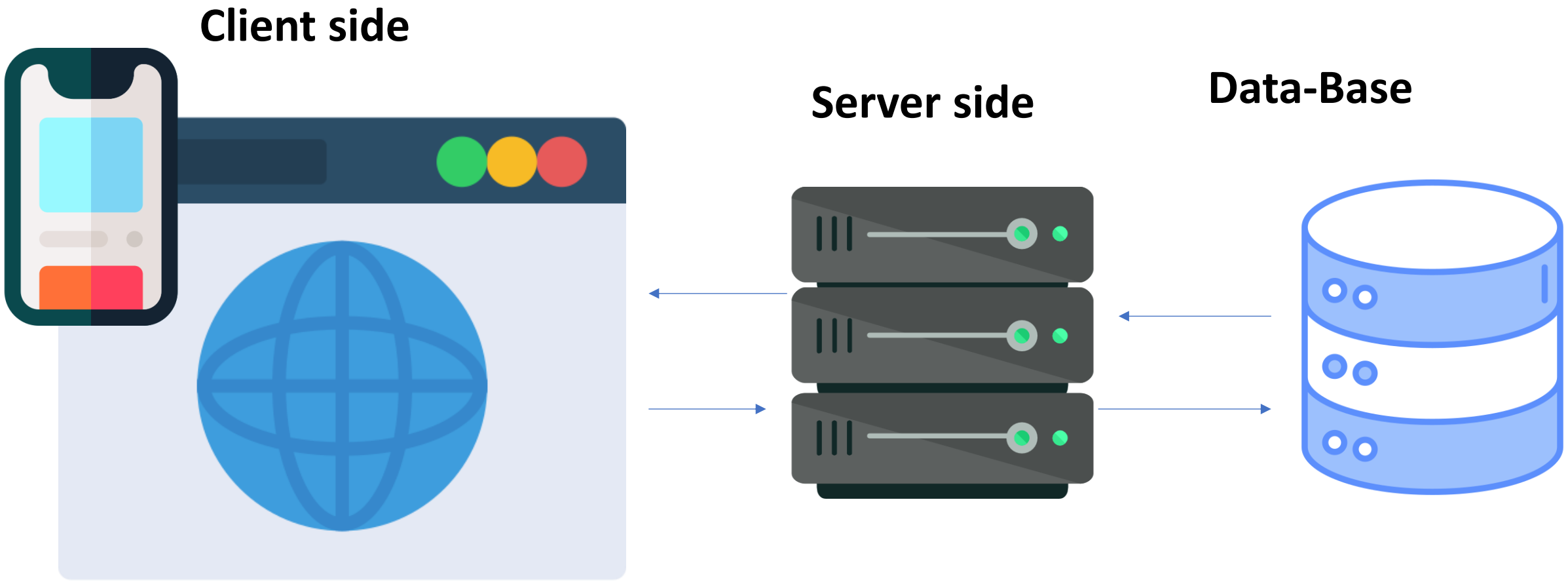
An <Outlet> should be used in parent route elements to render their child route elements. This allows nested UI to show up when child routes are rendered.

Hazem Alsaqaan

```
1
2  function Layout() {
3    return (
4      <>
5        <Header />
6        <Outlet />
7      </>
8    );
9  }
10
11 export default Layout;
12
```

Http Requests

(Axios)


Client side
Server side
Data-Base

HTTP Request and Methods with Example

Get

Retrieves data or resource from a specified URL. It is used to retrieve information without modifying it.

Post

Submit data or creates a new resource on the server. It is used to send data to be processed by the server. It often results in the creation of a new resource on the server.

Put

Updates the existing resource with the new data. It replaces the entire resource or creates it if it does not exist.

Delete

Deletes the specified resource from the server.

Patch

Partially updates the existing resource with the provided data. Partial update means a specific field (e.g., changing only the email address) of a user's profile. PATCH request does not create a new resource if the specified resource does not exist on the server.

Axios

Axios is basically an external library, which is used to make promise-based HTTP calls

```
npm install axios
```

Creating an instance for shared config :

```
import axios from 'axios';

export const instance = axios.create({
  baseURL: 'API BASE URL',
});
```

React Hooks

useEffect( )

Problem / Side effect:

- Fetch request runs on every render → high network usage.
- Server may get overloaded.
- Can cause infinite loop if setState triggers re-render.

Without useEffect

Advantages:

- Data is fetched only once.
- No unnecessary network requests.

With useEffect

Conclusion

- Without useEffect: code outside runs on every render, causing problems like API over fetching, memory leaks, and performance issues.
- With useEffect: code runs at the right time, only when needed. You can control when it executes (once, or when a state/prop changes). It also allows cleanup to prevent issues.

componentDidMount → when the component first mounts. [ ]

componentDidUpdate → when the component updates (state/props change). [**dep**]

componentWillUnmount → when the component unmounts (cleanup). **Return ( )=>**

React Hooks: useEffect

- Declare an effect. By default, your effect will run after every render.
- Specify the effect dependencies. Most effects should only rerun when needed rather than after every render. For example, a fade-in animation should only trigger when a component appears. Connecting and disconnecting to a chat room should only happen when the component appears and disappears or when the chat room changes. You will learn how to control this by specifying dependencies.
- Add cleanup if needed. Some effects need to specify how to stop, undo, or clean up whatever they were doing. For example, “connect” needs "disconnect," “subscribe” needs "unsubscribe," and “fetch” needs either “cancel” or "ignore." You will learn how to do this by returning a cleanup function.

https://www.freecodecamp.org/news/react-useeffect-absolute-beginners/

React Hooks: useEffect

Every React component goes through the same lifecycle:

- A component mounts when it’s added to the screen.
- A component updates when it receives new props or state, usually in response to an interaction.
- A component unmounts when it’s removed from the screen.

React Hooks: useEffect

useEffect : accepts a function that can perform any side effects.

Usage :

Hazem Alsaqaan

```
useEffect(() => {
   return () => {
     // clean your code on unmount component
   };
 }, [arrayDependencies]);
```

[]: empty array means to fire on mount only

[dependencies]: means to fire on mount and with every change in this dependency value

React Hooks: useRef

useRef :

It returns a ref object with a .current property. The ref object is mutable. Refs let a component hold some information that isn’t used for rendering, like a DOM node or a timeout ID. Unlike with state, updating a ref does not re-render your component.

Usage :

Hazem Alsaqaan

```
import { useRef } from "react";

function App() {
  const inputRef = useRef();

  return <input ref={inputRef} />;
}

export default App;
```

Thank you

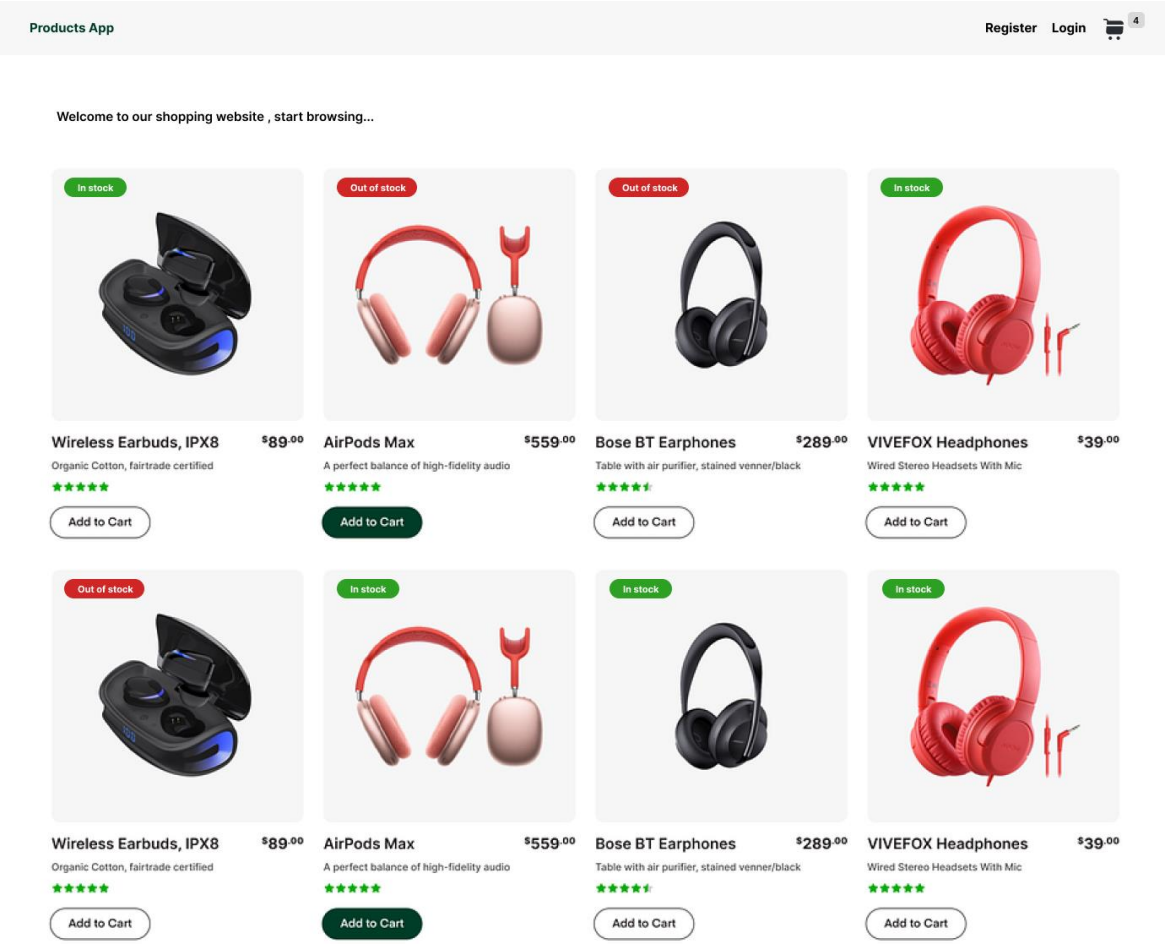
lap

Task: Ecommerce App

1: Add navbar with links for different pages as shown in task (products list, Product details, cart , not found)

2: Products list screen will include products list, using

this API https://dummyjson.com/products

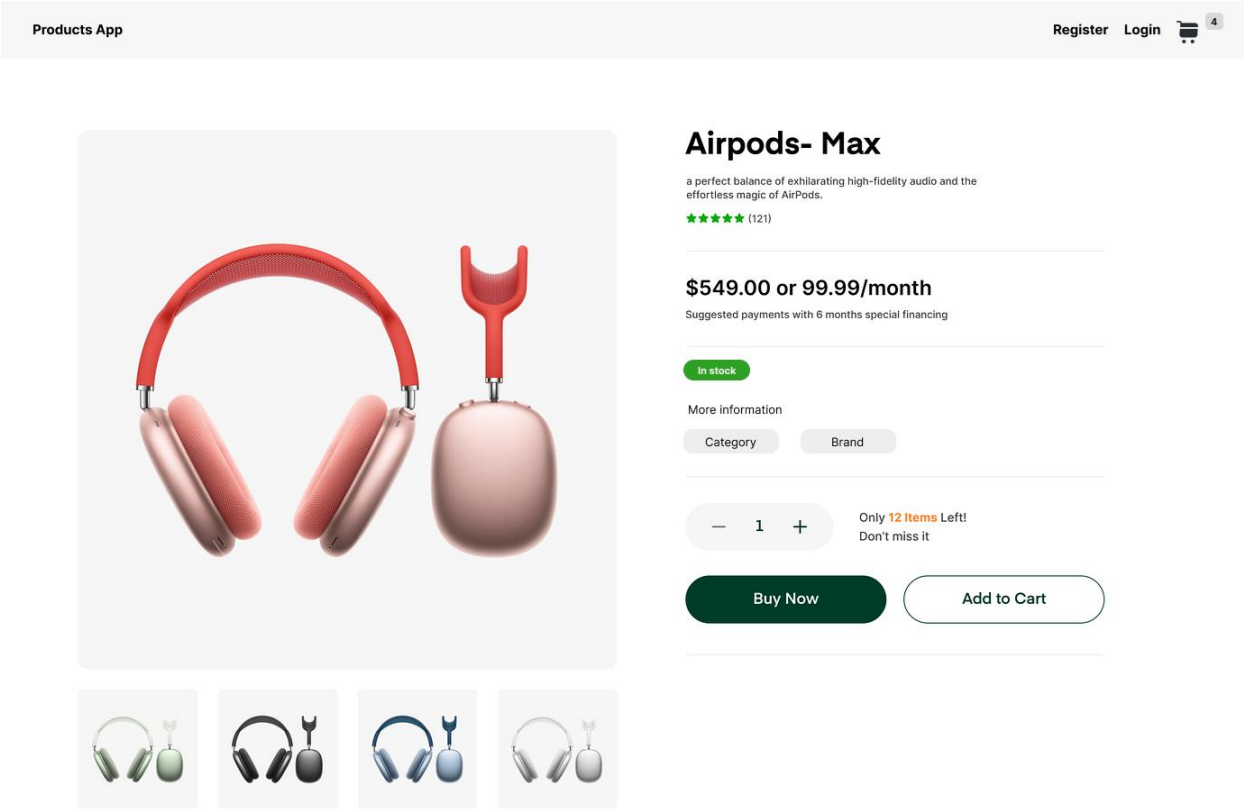


Task: Ecommerce App

3: When selecting a product, it should redirect to the product details screen and API to get data:

https://dummyjson.com/products/:id

4: **[Bonus]** Handle pagination for the products list using skip and limit to show 10 items per page.

Docs link for API:

https://dummyjson.com/docs/products#products-limit_skip